Savitribai Phule Pune University Second Year of Computer Engineering - 2024 Pattern		
PCC-253-COM : Computer Organization and Microprocessor		
Teaching Scheme	**Credits**	**Examination Scheme**
Theory : 02 Hours/Week	02	**CCE :** 30 Marks **End-Semester:** 70 Marks

UNIT I

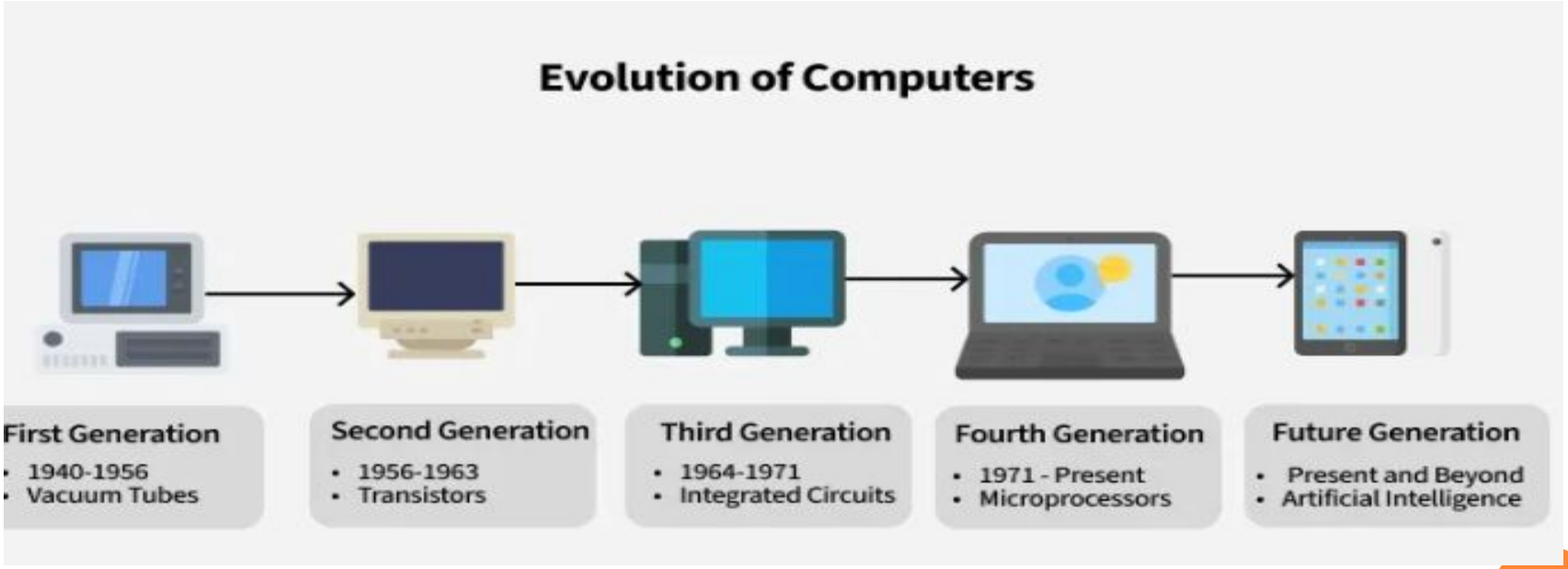
Computer Evolution and Performance

Contents

- A Brief History of Computers
- Von Neumann Architecture, Harvard Architecture, Designing for Performance
- Evolution of Intel processor architecture- 4 bit to 64 bit, performance assessment
- A top level view of Computer function and interconnection Computer Components
- Interconnection structure, bus interconnection
- Computer Arithmetic- The Arithmetic and Logic Unit, addition and subtraction of signed numbers, design of adder and fast adder, carry look ahead addition, multiplication of positive numbers, signed operand multiplication, Booths algorithm for multiplying binary integers.

A Brief History of Computers

- The history of computers spans thousands of years, from early counting devices to the powerful systems we use today. Here's an overview of the key milestones in the evolution of computers:



First Generation Computers

- In the period of the year 1940-1956, it was referred to as the period of the first generation of computers.
- These machines are slow, huge, and expensive.
- In this generation of computers, **vacuum tubes** were used as the basic components of CPU and memory.
- Also, they were mainly dependent on the batch operating systems and punch cards.
- Magnetic tape and paper tape were used as output and input devices.

Second Generation Computers

- he **were** period of the year, 1957-1963 was referred to as the period of the second generation of computers.
- It was the time of the transistor computers.
- In the second generation of computers**, transistors** (which were cheap in cost) are used.
- Transistors are also compact and consume less power. Transistor computers are faster than first-generation computers.
- For primary memory, magnetic cores were used, and for secondary memory magnetic disc and tapes for storage purposes.
- In second-generation computers, COBOL and FORTRAN are used as Assembly language and programming languages, and Batch processing and multiprogramming operating systems are allowed in these computers.
- For example **IBM 1620, IBM 7094, CDC 1604, CDC 3600**, etc.

Third Generation Computers

- In the third generation of computers, integrated circuits (ICs) were used instead of transistors(in the second generation).
- A single IC consists of many transistors which increased the power of a computer and also reduced the cost.
- The third generation computers are more reliable, efficient, and smaller in size. It used remote processing, time-sharing, and multiprogramming as operating systems. FORTRON-II TO IV, COBOL, and PASCAL PL/1 were used which are high-level programming languages.
- For example **IBM-360 series, Honeywell-6000 series, IBM-370/168,** etc.

Fourth Generation Computers

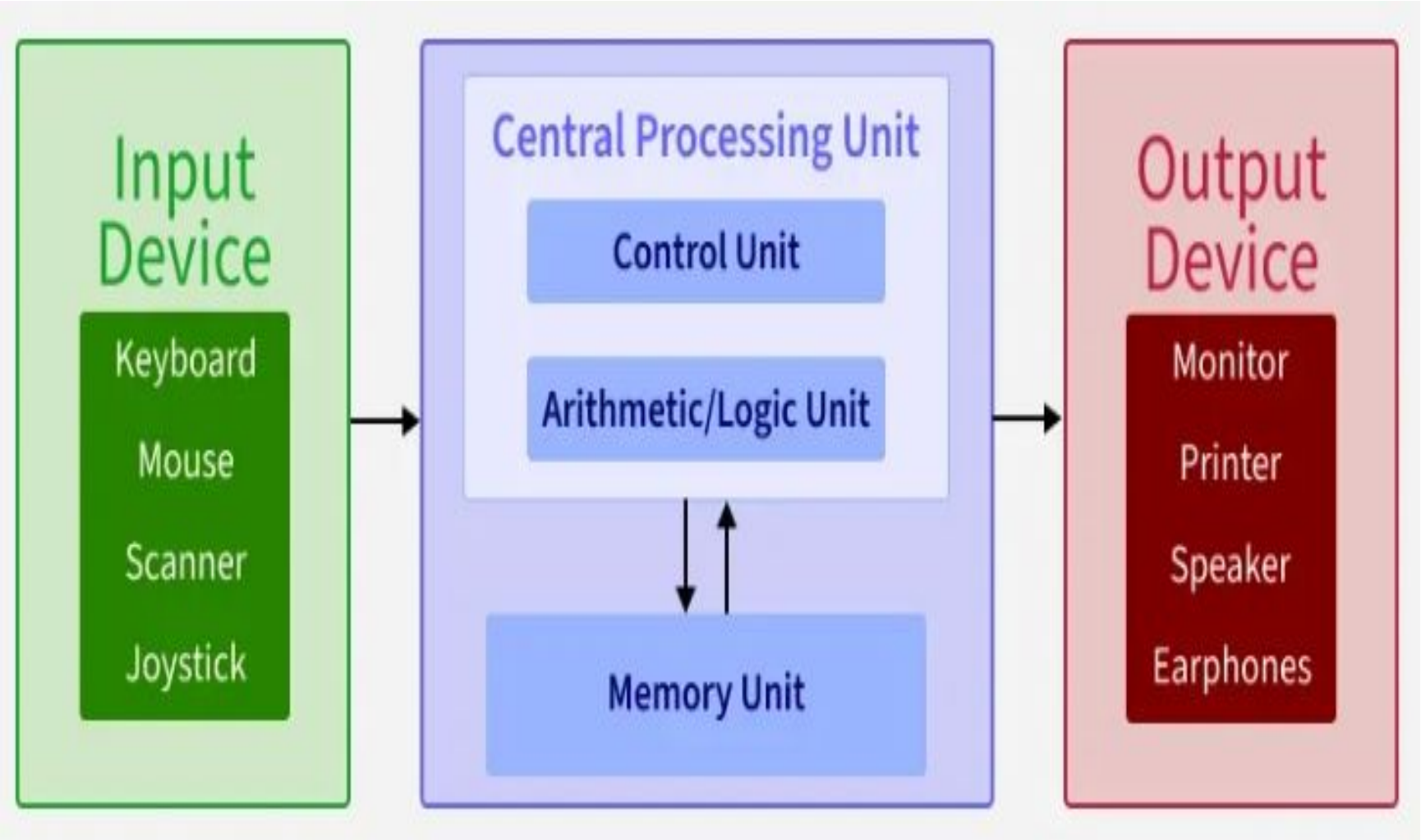
- The period of 1971-1980 was mainly the time of fourth generation computers.
- It used VLSI(Very Large Scale Integrated) circuits. VLSI is a chip containing millions of transistors and other circuit elements and because of these chips, the computers of this generation are more compact, powerful, fast, and affordable(low in cost).
- Real-time, time-sharing and distributed operating system are used by these computers.
- C and C++ are used as the programming languages in this generation of computers.
- For example **STAR 1000, PDP 11, CRAY-1, CRAY-X-MP,** etc.

Fifth Generation Computers

- From 1980 - to till date these computers have been **Internet** used.
- The ULSI (Ultra Large Scale Integration) technology is used in fifth-generation computers instead of the VLSI technology of fourth-generation computers.
- Microprocessor chips with ten million electronic components are used in these computers.
- Parallel processing hardware and AI (Artificial Intelligence) software are also used in fifth-generation computers. The programming languages like C, C++, Java, .Net, etc. are used.
- For example **Desktop, Laptop, NoteBook, UltraBook**, etc.

Von Neumann Architecture

- Von Neumann Architecture is a digital computer architecture whose design is based on the concept of stored program computers where program data and instruction data are stored in the same memory.
- This architecture was designed by the famous mathematician and physicist **John Von Neumann** in 1945.

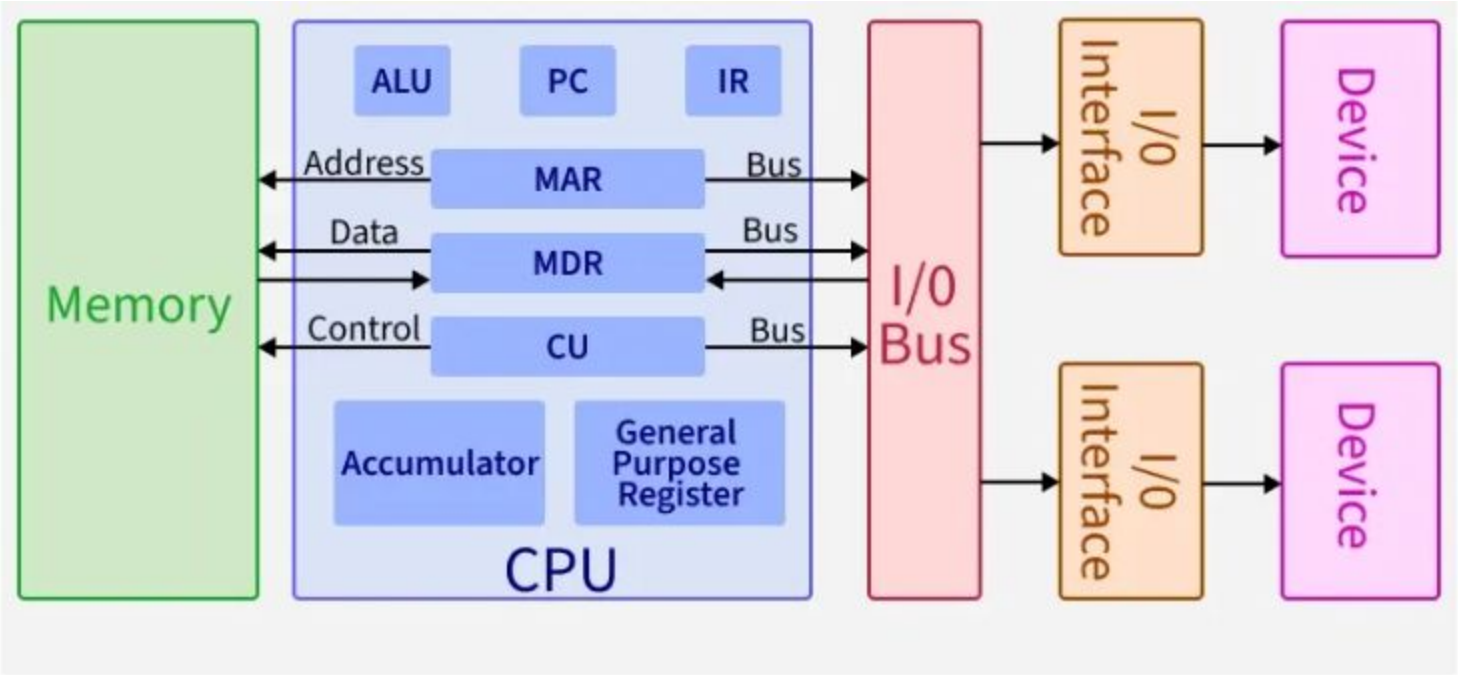

Input Device
Keyboard
Mouse
Scanner
Joystick
Central Processing Unit
Control Unit
Arithmetic/Logic Unit
Memory Unit
Output Device
Monitor
Printer
Speaker
Earphones

Components of Von Neumann Architecture

- The structure described in the figure outlines the basic components of a computer system, particularly focusing on the memory and processor. It is made up with three main components:
- CPU
- Memory
- I/O Devices

CPU (Central Processing Unit)

- The central processing unit (CPU) is the main part of a computer that controls how it works. It is made up of the control unit, main memory, and the arithmetic logic unit (ALU). The CPU handles instructions from programs, processes data, stores information, and produces results. Without the CPU, a computer cannot perform tasks or run any applications.



Basic CPU structure, illustrating ALU

- ***CU (Control Unit)***
- The control unit manages how the processor works by sending control signals. It decides how data should move inside the computer, controls input and output operations, and fetches the instructions from memory for execution.
- ***ALU (Arithmetic and Logic Unit)***
- The arithmetic and logic unit is the part of the CPU that handles the calculations and decision-making tasks. It performs arithmetic operations like addition and subtraction, logical operations such as comparisons, and tasks like shifting bits in data.
- ***Registers***
- Registers are the fastest type of memory located inside the CPU. They temporarily store information that the processor is currently working on, making program execution and operations faster and more efficient. Register serve as the CPU's primary working memory.
- **PC (Program Counter)**: Keeps track of the address of the next instruction to be executed.
- **IR (Instruction Register)**: Holds the current instruction being executed.
- **MAR (Memory Address Register)**: Stores the address of the memory location being accessed.
- **MDR (Memory Data Register)**: Temporarily holds data being transferred to or from memory.
- **Accumulator**: A register that stores intermediate results of arithmetic and logic operations.
- **General Purpose Registers**: Used for temporary storage of data during processing.

- ***Bus***
- The bus is a communication system that transfers data, addresses, and control signals between the CPU, memory, and I/O devices. In Von Neumann architecture, a single bus is shared for both data and instructions, which can create a bottleneck (known as the Von Neumann bottleneck).
- ***I/O Bus***

  I/O Interface: Connects the CPU and memory to input/output devices.

  Device: Refers to external hardware like keyboards, monitors, or storage devices.
- ***Key Characteristics***
- **Single Memory for Data and Instructions**: Both data and program instructions are stored in the same memory.
- **Shared Bus**: A single bus is used for transferring data, addresses, and control signals, which can limit performance.
- **Sequential Execution**: Instructions are executed one at a time in a sequential manner.

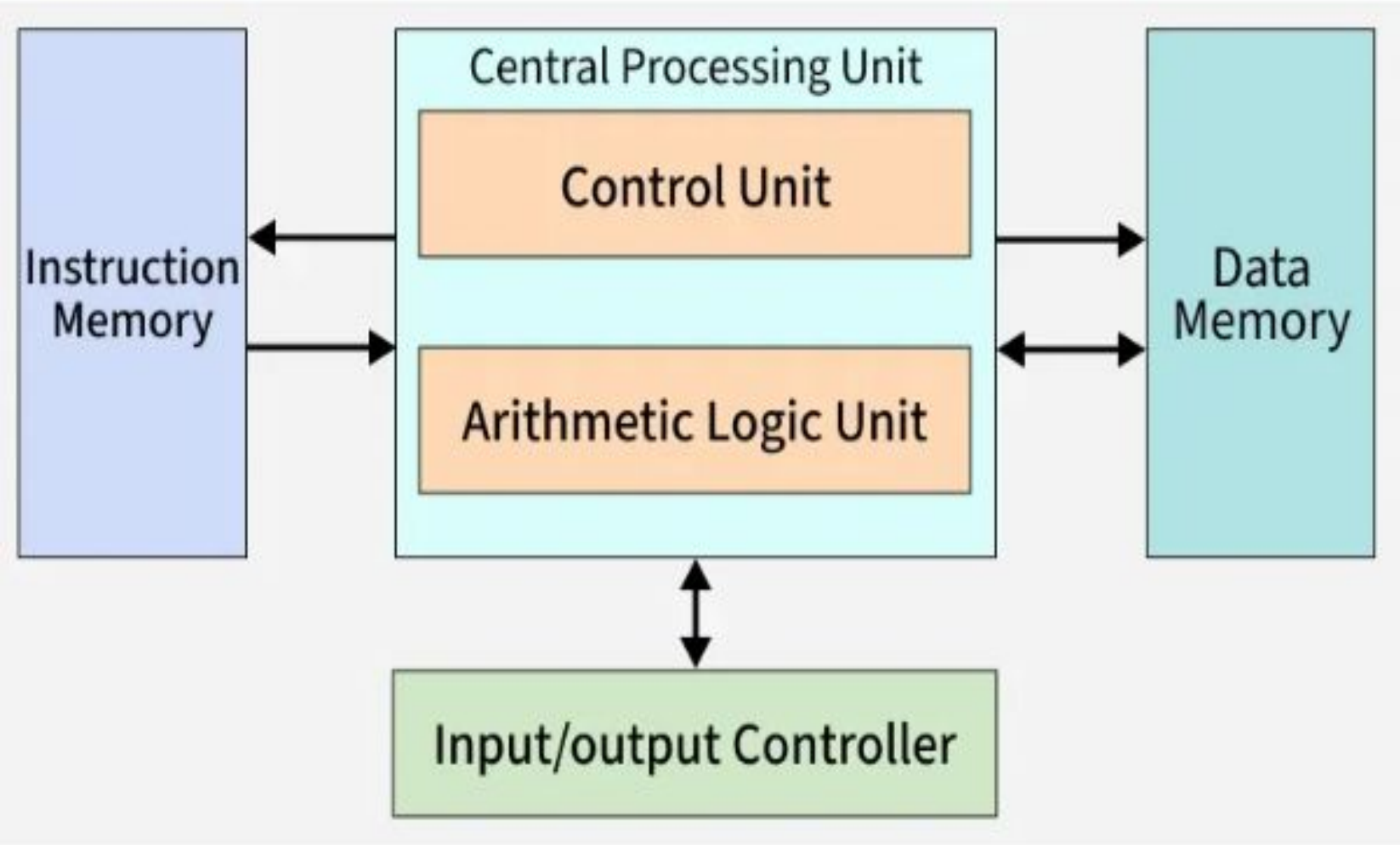
- ***Advantages of Von Neumann Architecture***
- **Simplicity:** The fact that all data and instructions are stored in a single memory space helps the process of designing a computer system as there is no need to create complicated systems of routing since pathways may coincide.
- **Cost-Effective:** A smaller number of components is needed as compared to the other architectural designs hence more economical.
- **Flexibility:** A program can Always be changed or altered without experiencing a change in some underlying physical aspects such as the circuitry.
- ***Disadvantages of Von Neumann Architecture***
- **Bottleneck Issues:** The shared bus can be a problem because the data and control instructions cannot be obtained simultaneously and therefore it becomes slow.
- **Memory Corruption:** Since the data and instructions reside in the same memory, then there is a temptation of one erasing the other thereby producing system faults.

Harvard Architecture

- Harvard architecture is a computer design model where program instructions and data are stored in separate memory units that are accessed through independent buses.
- This separation allows the processor to fetch instructions and access data simultaneously, which helps avoid the bottleneck present in traditional Von Neumann systems.
- Eliminates the Von Neumann bottleneck.
- Faster and predictable performance (suitable for real-time systems).
- Parallel access to both instructions and data.

Working Principle

- In Harvard Architecture, fetching an instruction from instruction memory and reading/writing data from/to data memory happen at the same time without waiting for one to finish.
- Separate buses prevent the bottleneck that occurs when data and instructions share a path.
- For example, while an instruction is being executed, the next instruction can be fetched simultaneously, speeding up processing.



Structure of Harvard Architecture

- ***Buses***
- Buses are used as signal pathways. In Harvard architecture, there are separate buses for both instruction and data. Types of Buses:
- **Data Bus:** It carries data among the main memory system, processor, and I/O devices.
- **Data Address Bus:** It carries the address of data from the processor to the main memory system.
- **Instruction Bus:** It carries instructions among the main memory system, processor, and I/O devices.
- **Instruction Address Bus:** It carries the address of instructions from the processor to the main memory system.
- ***Components of Harvard Architecture***
- Harvard architecture is designed with specific components that handle instruction execution, control, and data communication.
- **Arithmetic and Logic Unit:** The arithmetic logic unit is part of the CPU that operates all the calculations needed. It performs addition, subtraction, comparison, logical Operations, bit Shifting Operations, and various arithmetic operations.
- **Control Unit:** The Control Unit is the part of the CPU that operates all processor control signals. It controls the input and output devices and also controls the movement of instructions and data within the system.
- **Input/ Output System:** Input devices are used to read data into main memory with the help of CPU input instruction. The information from a computer as output is given through Output devices. The computer gives the results of computation with the help of output devices.

Application of Harvard Architecture

- Harvard architecture is a type of computer design where the memory for instructions and data are kept separate. Here are the some applications:
- **Digital Signal Processors (DSPs):**
- Audio and video processing, telecommunications, radar systems, and image processing.
- Texas Instruments TMS320 for hearing aids.
- **Microcontrollers (MCUs):**
- Embedded systems in consumer electronics, automotive systems, IoT devices, and industrial automation**.**
- PIC in automotive ABS
- **Network Processors:**
- Routers, switches, and network security appliances.
- Broadcom StrataXGS
- **Automotive Systems:**
- Engine control units (ECUs), advanced driver-assistance systems (ADAS), and infotainment systems.
- NXP S32K for engine control

VON NEUMANN ARCHITECTURE	HARVARD ARCHITECTURE
It is ancient computer architecture based on stored program computer concept.	It is modern computer architecture based on Harvard Mark I relay based model.
Same physical memory address is used for instructions and data.	Separate physical memory address is used for instructions and data.
There is common bus for data and instruction transfer.	Separate buses are used for transferring data and instruction.
Two clock cycles are required to execute single instruction.	An instruction is executed in a single cycle.
It is cheaper in cost.	It is costly than Von Neumann Architecture.
CPU can not access instructions and read/write at the same time.	CPU can access instructions and read/write at the same time.
It is used in personal computers and small computers.	It is used in micro controllers and signal processing.

Designing for Performance,

- In computer organization, performance refers to the speed and efficiency at which a computer system can execute tasks and process data. A high-performing computer system is one that can perform tasks quickly and efficiently while minimizing the amount of time and resources required to complete these tasks.
- Here are several factors that can impact the performance of a computer system, including:
- **Processor speed:** The speed of the processor, measured in GHz (gigahertz), determines how quickly the computer can execute instructions and process data.
- **Memory:** The amount and speed of the memory, including RAM (random access memory) and cache memory, can impact how quickly data can be accessed and processed by the computer.
- **Storage:** The speed and capacity of the storage devices, including hard drives and solid-state drives (SSDs), can impact the speed at which data can be stored and retrieved.
- **I/O devices:** The speed and efficiency of input/output devices, such as keyboards , mice, and displays, can impact the overall performance of the system.
- **Software optimization:** The efficiency of the software running on the system, including operating systems and applications, can impact how quickly tasks can be completed.

- **Computer performance** is the amount of work accomplished by a computer system. The word performance in computer performance means "How well is the computer doing the work it is supposed to do?". It basically depends on the response time, throughput, and execution time of a computer system.
- **Response time** is the time from the start to completion of a task. This also includes:
- Operating system overhead.
- Waiting for I/O and other processes
- Accessing disk and memory
- Time spent executing on the CPU or execution time.

- **Throughput** is the total amount of work done in a given time. **CPU execution time** is the total time a CPU spends computing on a given task. It also excludes time for I/O or running other programs. This is also referred to as simply CPU time. Performance is determined by execution time as performance is inversely proportional to execution time.
- Performance = (1 / Execution time)
- (Performance of A / Performance of B)
  = (Execution Time of B / Execution Time of A)
- If given that Processor A is faster than processor B, that means execution time of A is less than that of execution time of B. Therefore, performance of A is greater than that of performance of B. **Example -** Machine A runs a program in 100 seconds, Machine B runs the same program in 125 seconds

- **How to Improve Performance?**
- To improve performance you can either:
- Decrease the CPI (clock cycles per instruction) by using new Hardware.
- Decrease the clock time or Increase clock rate by reducing propagation delays or by use pipelining.
- Decrease the number of required cycles or improve ISA or Compiler.

Evolution of Intel processor architecture- 4 bit to 64 bit

Transistor was invented in 1948 (23 December 1947 in Bell lab). IC was invented in 1958 (Fair Child Semiconductors) By Texas Instruments J Kilby. The first microprocessor was invented by INTEL(INTegrated ELectronics).

Size of the microprocessor - 4 bit

Name	Year of Invention	Clock speed	Number of transistors	Inst. per sec
INTEL 4004/4040	1971 by Ted Hoff and Stanley Mazor	740 kHz	2300	60,000

Size of the **microprocessor -** 8 bit

Name	Year of Invention	Clock speed	Number of transistors	Inst. per sec
8008	1972	500 kHz	3500	50,000
8080	1974	2 MHz	6000	10 times faster than 8008
8085	1976 (16-bit address bus)	3 MHz	6500	769230

Size of the microprocessor - 16 bit

Name	Year of Invention	Clock speed	Number of transistors	Inst. per sec
8086	1978 (multiply and divide instruction, 16-bit data bus and 20-bit address bus)	4.77 MHz, 8 MHz, 10 MHz	29000	2.5 Million
8088	1979 (cheaper version of 8086 and 8-bit external bus)			2.5 Million
80186/80188	1982 (80188 cheaper version of 80186, and additional components like interrupt controller, clock generator, local bus controller, counters)	6 MHz		
80286	1982 (data bus 16bit and address bus 24 bit)	8 MHz	134000	4 Million

Size of the microprocessor - 32 bit

Name	Year of Invention	Clock speed	Number of transistors	Inst. per sec
INTEL 80386	1986 (other versions 80386DX, 80386SX, 80386SL , and data bus 32-bit address bus 32 bit)	16 MHz - 33 MHz	275000	
INTEL 80486	1986 (other versions 80486DX, 80486SX, 80486DX2, 80486DX4)	16 MHz - 100 MHz	1.2 Million transistors	8 KB of cache memory
PENTIUM	1993	66 MHz		Cache memory 8 bit for instructions 8 bit for data

Size of the microprocessor - 64 bit

Name	Year of Invention	Clock speed	Number of transistors	Inst. per sec
INTEL core 2	2006 (other versions core2 duo, core2 quad, core2 extreme)	1.2 GHz to 3 GHz	291 Million transistors	64 KB of L1 cache per core 4 MB of L2 cache
i3, i5, i7	2007, 2009, 2010	2.2GHz - 3.3GHz, 2.4GHz - 3.6GHz, 2.93GHz - 3.33GHz		

- **Generations of microprocessors:**
- **First-generation -**
  From 1971 to 1972 the era of the first generation came which brought microprocessors like INTEL 4004 Rockwell international PPS-4 INTEL 8008 etc.

- **Second generation -**
  The second generation marked the development of 8-bit microprocessors from 1973 to 1978. Processors like INTEL 8085 Motorola 6800 and 6801 etc came into existence.

- **Third generation -**
  The third generation brought forward the 16-bit processors like INTEL 8086/80186/80286 Motorola 68000 68010 etc. From 1979 to 1980 this generation used the HMOS technology.

- **Fourth generation -**
  The fourth-generation came into existence from 1981 to 1995. The 32-bit processors using HMOS fabrication came into existence. INTEL 80386 and Motorola 68020 are some of the popular processors of this generation.

- **Fifth-generation -**
  From 1995 till now we are in the fifth generation. 64-bit processors like PENTIUM, Celeron, dual, quad, and octa-core processors came into existence.

Performance measurement

- Performance measurement in computer organization is the evaluation of a computer system's efficiency and effectiveness using metrics like **execution time, throughput, and latency**.
- Other key measures include processor speed (clock speed), cycles per instruction (CPI), and floating-point operations per second (FLOPS).
- These measurements help identify bottlenecks, optimize system design, and compare different architectures using tools like benchmarks.

Key performance metrics

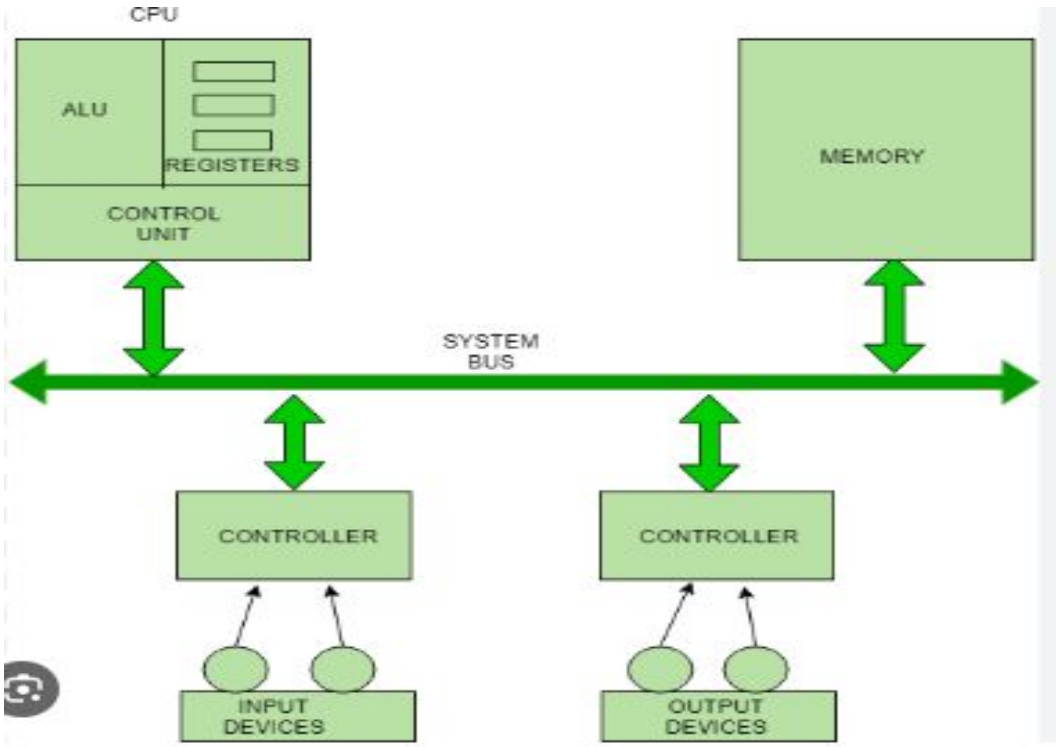
- **Execution time:** The time it takes for a computer to complete a single task. The formula is: Execution Time = Instructions × CPI × Clock Cycle Time.
- **Throughput:** The number of jobs or tasks a computer can complete in a given amount of time.
- **Latency:** The time it takes to complete a specific operation or task.
- **Processor speed (Clock speed):** Measured in gigahertz (GHz), this indicates how quickly a processor can execute instructions.
- **Cycles per Instruction (CPI):** The average number of clock cycles a CPU requires to complete one instruction.
- **FLOPS (Floating-point Operations Per Second):** A measure of how many floating-point calculations a system can perform per second, often used for scientific computations.

How measurements are used

- **Identify bottlenecks:** Measurements help pinpoint performance limitations within the system, such as a slow CPU or memory.
- **Optimize design:** By understanding the performance of different components, designers can make changes to improve overall efficiency.
- **Compare architectures:** Metrics and standardized benchmarks (like HPL for High-Performance Computing) allow for objective comparisons between different computer systems.

A top level view of Computer function and interconnection Computer Components

- A computer's top-level view consists of a **Central Processing Unit (CPU)**, **memory**, and **input/output (I/O)** devices, all connected by an **interconnection structure**, most commonly a bus system. The CPU executes instructions, memory stores data and instructions, I/O devices allow for interaction with the outside world, and the interconnection structure, often a bus, provides pathways for data and control signals to travel between these components.



- **Computer components and their functions**
- **Central Processing Unit (CPU):** The "brain" of the computer, responsible for executing instructions and processing data. It contains the control unit and the arithmetic logic unit (ALU).
- **Memory:**
  - **RAM (Random Access Memory):** Temporary, high-speed storage for data and instructions currently being used by the CPU.
- **ROM (Read-Only Memory):** Permanent storage for firmware that cannot be easily changed.
- **Input/Output (I/O) Devices:Input devices:** Hardware that sends data and instructions to the computer, such as keyboards and mice.
- **Output devices:** Hardware that presents the results of the computer's processing, like monitors and printers.
- **Storage Devices:** Long-term persistent storage for data, including hard drives and SSDs.**Motherboard:** The main circuit board that physically connects all the components together.**Power Supply Unit (PSU):** Converts AC power from the wall into the DC power needed to run the system.

- **The interconnection structure**
- **Buses:** The main pathways that connect the CPU, memory, and I/O modules.
  - **Data Bus:** Provides a path for moving data between components. Its "width" (number of lines) determines how many bits can be transferred at once.
  - **Address Bus:** Specifies the source or destination of data.
  - **Control Bus:** Carries control and timing signals that manage the operations of all components.
- **Direct Memory Access (DMA):** Allows I/O devices to directly transfer data to and from memory without involving the CPU, which can improve performance for certain operations.

INTERCONNECTIONS OF UNITS..

- Set of wires used for interconnection is known as system bus which carry group of bits (information) in a controlled manner.
- It is further divided into three logical units, namely the address bus, the data bus, and the control bus.

System Buses Types

- Data Bus: The data bus is used when any unit is sending data, instruction or command code to some other units.
- Address Bus: The address bus is used when one unit is sending an address information i.e. location of the data residing in the memory to another unit.

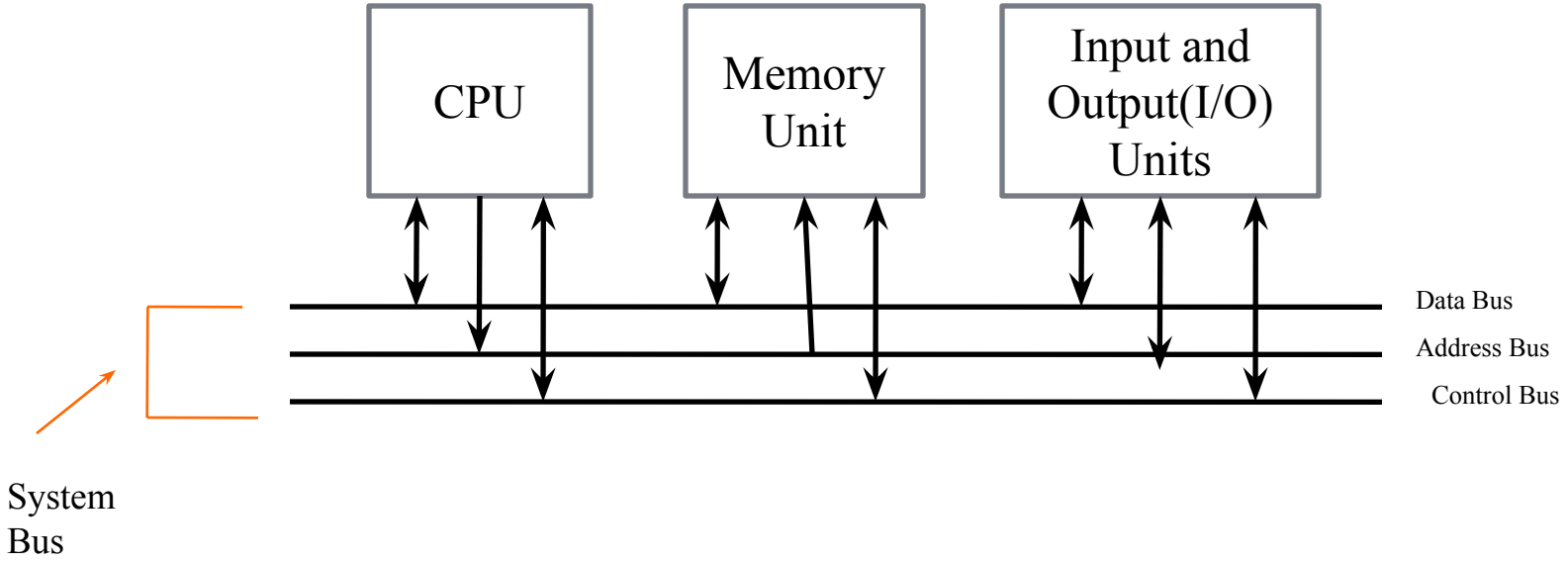
Control Bus

- Control Bus: The control bus is responsible for making CPU, memory and I/O devices work together as a functional system, carrying signals that report the status (ready, not ready) of various units.
- The function of a control bus is to determine and instruct according to the operation type (Read or Write). For example, if the processor or an I/O device wants to read or write a value from memory, the control bus will specify it.

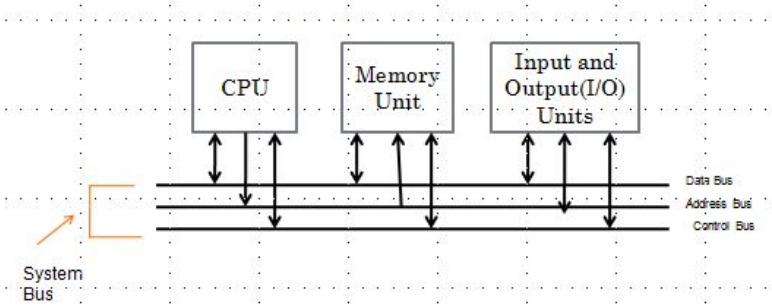
Processing of Information

- The bus is common to all the units in the computer. Before sending some information on the bus, an unit should verify whether the bus is free or occupied with some communication started by some other unit.
- CPU is the bus master in a computer which decides who should control the bus when more than one unit wants the bus at the same time.
- An unit who needs the bus makes a request to the CPU and waits sanction. Till the CPU issues sanction, the requesting unit does not attempt to use the bus.

Interconnection of Computers Units via System Bus

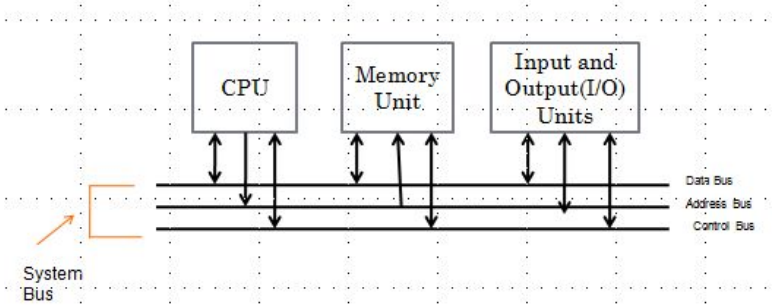


Interconnection of Computers Units via Bus



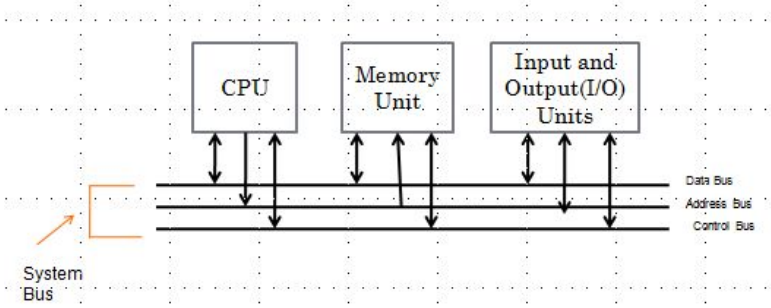
- Shows how the system bus interconnects the processor, memory and I/O devices.
- Both processor and memory units hold a bi-directional relationship with the control and data bus.

Interconnection of Computers Units via Bus



- In case of an address bus, the communication with processor and memory is unidirectional.
- Processor provides location of data (stored in the register) to be fetched from the memory to the address bus and the data carries the required data to the processor.

Interconnection of Computer Units via BUS



- I/O devices have a bi-directional relationship with the system bus.

- **Computer Functions**
- At the most basic level, a computer performs four main functions as part of its program execution cycle:
- **Input:** Receiving data and instructions from the external environment via input devices.
- **Processing:** The CPU performs arithmetic and logical operations on the data according to the instructions.
- **Storage:** Storing data and instructions temporarily in main memory (RAM) or permanently in secondary storage devices.
- **Output:** Presenting the processed data (results) to the user through output devices.
- The CPU continuously repeats an **instruction cycle** to achieve these functions, which involves fetching an instruction from memory, decoding it, and then executing it. This cycle is managed by the control unit and relies heavily on the interconnection system to move data and signals between the various components efficiently.

Interconnection structure, bus interconnection

- An interconnection structure is the collection of paths that connect the modules of a computer, such as the CPU, memory, and I/O devices.
- A bus interconnection is a common type of structure where a shared communication pathway, called a bus, connects multiple devices.
- Buses consist of data, address, and control lines, and only one device can transmit at a time to avoid signal overlap

Bus interconnection structure

- **Definition**: A bus is a shared transmission medium that connects two or more devices, allowing them to communicate.
- **Components**: A bus typically includes three types of lines:
  - **Data lines**: Transmit the actual data to be moved between components.
  - **Address lines**: Specify the source or destination of the data.
  - **Control lines**: Carry command and timing signals to manage the bus and data transfer process.
- **Function**: Multiple devices connect to the bus, but only one device can transmit at a time.
- When a device wants to communicate, it first checks if the bus is free.
- The sending device places the destination address on the address lines and a command on the control lines.
- The destination device identifies the address and performs the requested operation.
- **System Bus**: The main bus that connects the major components of a computer (processor, memory, and I/O) is called the system bus.
- **Performance**: The width of the data bus (number of bits transferred at once) impacts system performance; a wider bus can transfer more data simultaneously.

Computer Arithmetic-
Arithmetic logic Unit

An arithmetic logic unit (ALU) is a combinational digital electronic circuit that performs arithmetic and bitwise operations on integer binary numbers. This is in contrast to a floating-point unit (FPU), which operates on floating point numbers. An ALU is a fundamental building block of many types of computing circuits, including the central processing unit (CPU) of computers, FPUs, and graphics processing units(GPUs). A single CPU, FPU or GPU may contain multiple ALUs.

Functions and Operations of ALU

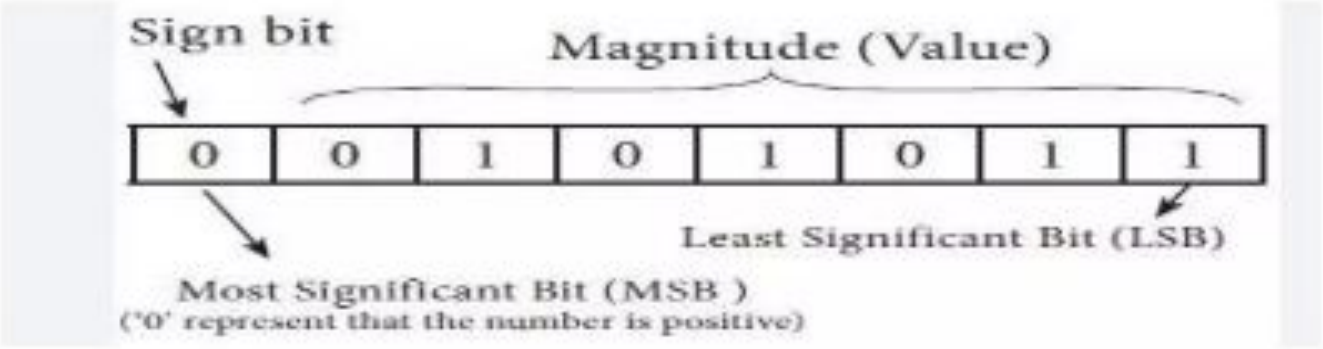
- **1. Arithmetic Operations**
- ALU performs arithmetic operations to carry out basic mathematical calculations on binary numbers.
- Operations include addition, subtraction, increment, and decrement.
- Used during instruction execution for tasks like ADD, SUB, INC, and DEC.
- **2. Logical Operations**
- These operations manipulate data at the bit level using logic gates.
- Includes bitwise operations like AND, OR, XOR, and NOT.
- Essential for masking, setting, clearing, or toggling specific bits.
- **3. Shift Operations**
- ALU can shift bits left or right to aid in fast arithmetic and bit manipulation.
- Types include logical shift, arithmetic shift, and rotate operations.
- Often used for multiplication/division by powers of two and bit-level data formatting.
- **4. Comparison Operations**
- ALU supports comparison by evaluating two operands and setting appropriate flags.
- It checks for conditions like equality, greater than, or less than.
- Useful in branching and conditional instructions based on flag values.
- **5. Status Flag Generation**
- The ALU sets flags based on the result of operations to influence program control.
- Common flags: Zero (Z), Carry (C), Sign (S), Overflow (V).
- Flags are used in conditional execution and exception handling.

- ***How it works***
- The **control unit** decodes an instruction and sends signals to the ALU, specifying which operation to perform.
- Data, called **operands**, is retrieved from registers within the CPU.
- The ALU performs the requested operation on the operands using its internal circuits, which are built using logic gates.
- The result of the operation is sent to a destination register or memory.
- The ALU also generates **status bits** (like carry, zero, negative, or overflow) to indicate properties of the result, which can be used in subsequent instructions.
- ***Components of an ALU***
- **Arithmetic Circuits:** Perform arithmetic operations like addition and subtraction.
- **Logic Circuits:** Perform logic operations like AND, OR, and XOR.
- **Control Circuits:** Coordinate the flow of data and operations within the ALU based on the control unit's instructions.

Addition and Subtraction with Signed –Magnitude Data

Fixed Point Number Representation

- The Fixed Point / Integer Numbers are represented in to two forms they are Signed Integer and Unsigned Integer.
- Signed Integer Number Represents Negative Numbers MSB 1
- Unsigned Integer Number Represents Positive Numbers MSB 0



- The left most bit (sign bit) in the binary number represents sign of the number.
- MSB is 1 the number represents negative number.
- MSB is 0 the number represents positive number.

Here are some examples of sign-magnitude numbers.

1. + 6	=	0	0	0	0	0	1	1	0
2. − 14	=	1	0	0	0	1	1	1	0
3. + 24	=	0	0	0	1	1	0	0	0
4. − 64	=	1	1	0	0	0	0	0	0

Maximum positive number	0 1 1 1	1 1 1 1 = + 127
Maximum negative number	1 1 1 1	1 1 1 1 = − 127

One's Complement Representation

- In the 1's complement representation, negative numbers are obtained by complementing each bit of the corresponding positive number.

Example 1.1 : *Find 1's complement of* $(1101)_2$.

Solution :

1	1	0	1	← number
0	0	1	0	← 1's complement

Example 1.2 : *Find 1's complement of* 1 0 1 1 1 0 0 1.

Solution :

1	0	1	1	1	0	0	1	← number
0	1	0	0	0	1	1	0	← 1's complement

1.2.3 Two's Complement Representation

The two's complement number is obtained by subtracting corresponding positive number from 2^n. Hence, the 2's complement number is obtained by adding 1 to the 1's complement number.

Example 1.3 : *Find 2's complement of* $(11000100)_2$.

Solution :

1	1	0	0	0	1	0	0	Number
					1	1		Carry
0	0	1	1	1	0	1	1	1's complement of Number
							1	Add 1
0	0	1	1	1	1	0	0	2's complement of Number

2.2.1 Addition

Rule for Addition

- Rules for binary addition are as follows :

$0 + 0 = 0$

$0 + 1 = 1$

$1 + 0 = 1$

$1 + 1 = 10_2$

Example : Add $(6)_{10}$ to $(7)_{10}$ in binary.

1	1			Carry
0	1	1	0	$= (6)_{10}$
0	1	1	1	$= (7)_{10}$
1	1	0	1	$= (13)_{10}$

- The first three operations produce a sum whose length is one digit, but when the last operation is performed sum is two digits. The higher significant bit of this result is called a **carry**, and lower significant bit is called **sum**.
- The logic circuit which performs this operation is called a **half-adder**. The circuit which performs addition of three bits (two significant bits and a previous carry) is a **full-adder**.

2.2.1.1 Half Adders

- The half-adder operation needs two binary inputs : augend and addend bits; and two binary outputs : sum and carry.
- The truth table shown in Table 2.2.1 gives the relation between input and output variables for half-adder operation.

Inputs		Outputs	
A	B	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Table 2.2.1 Truth table for half-adder

Fig. 2.2.1 Block schematic of half-adder

K-map simplification for carry and sum

For carry

A \ B	0	1
0	0	0
1	0	1

Carry = AB

For sum

A \ B	0	1
0	0	1
1	1	0

Sum = $A\overline{B} + \overline{A}B$
= $A \oplus B$

Fig. 2.2.2 Maps for half-adder

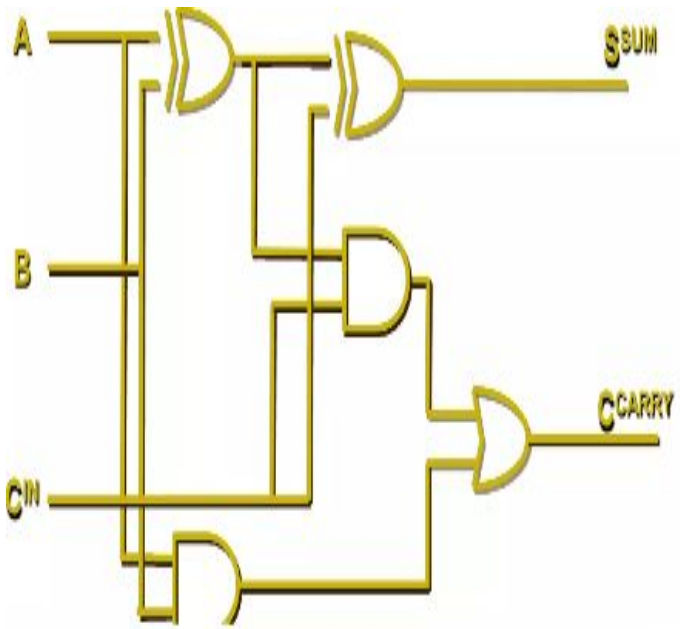
Logic diagram

Fig. 2.2.3 Logic diagram for half-adder

WHAT IS FULL ADDER

- The full adder accepts two inputs bits and an input carry and generates a sum output and an output carry.
- The full-adder circuit adds three one-bit binary numbers (Cin, A ,B) and outputs two one-bit binary numbers, a sum (S) and a carry (Cout). The full-adder is usually a component in a cascade of adders, which add 8, 16, 32, etc. binary numbers.

Circuit Diagram for Full Adder



•Three input

A, B, C_{carry}

Two output

C_{out}, Sum

Truth Table

INPUTS			OUTPUTS	
A	B	C^{IN}	C^{CARRY}	S^{SUM}
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Binary Arithmetic - Negative Numbers in 2's Complement form in the

Case 1: (Both Positive) : Add $(28)_{10}$ and $(15)_{10}$

$(28)_{10} = \boxed{0}11100_2$

$(15)_{10} = \boxed{0}1111_2$

	16	8	4	2	1	
	$\boxed{0}$ $\boxed{0}$	1	1	1	0	0
+	$\boxed{0}$ $\boxed{0}$ $\boxed{0}$	1	1	1	1	
	$\boxed{0}$	1	0	1	0	1 1

$$\begin{array}{r} +28 \\ +15 \\ \hline (43)_{10} \end{array}$$

Example

$0 + 0 = 0$

$0 + 1 = 1$

$1 + 0 = 1$

$1 + 1 = 10$ (which is 0 Carry 1)

SIGNED BIT REPRESENTATION

Representation of both *positive* and *negative* numbers

- Following 3 representations

Signed magnitude representation
Signed 1's complement representation
Signed 2's complement representation

Example: Represent +9 and -9 in 7 bit-binary number

Only one way to represent + 9 ==> 0 001001

Three different ways to represent - 9:

In signed-magnitude:	1 001001
In signed-1's complement:	1 110110
In signed-2's complement:	1 110111

Sign-magnitude number

- A sign-magnitude number Z can be represented as (A_s, A) where A_s is the sign of Z and A is the magnitude of Z.
- The leftmost position, A_s, is the sign bit.
- The sign bit is either positive = 0 or negative = 1

Number	Signed-Magnitude
+3	0 11
+2	0 10
+1	0 01
+0	0 00
-0	1 00
-1	1 01
-2	1 10
-3	1 11

ADDITION ALGORITHM

- When the sign of A and B are same, add the magnitudes and attach the sign of A to the result.

- Otherwise compare the magnitudes and subtract the smaller number from the larger.
  - Choose the sign of result to be same as A if A>B
  - or the complement of sign of A if A<B
  - if A=B subtract B from A and make the sign of result positive

SIGNED BIT ADDITION

Operation	Add Magnitudes	Subtract Magnitudes		
		A>B	A<B	A=B
(+A)+(+B)	+(A+B)			
(-A)+(-B)	-(A+B)			
(+A)+(-B)		+(A-B)	-(B-A)	+(A-B)
(-A)+(+B)		-(A-B)	+(B-A)	+(A-B)

Example 1

Let us perform the **addition** of two decimal numbers **+7 and +4** using 2's complement method.

The **2's complement** representations of +7 and +4 with 5 bits each are shown below.

$$+7_{10} = 00111_2$$

$$+4_{10} = 00100_2$$

The addition of these two numbers is

$$+7_{10} + +4_{10} = 00111_2 + 00100_2$$

$$\Rightarrow +7_{10} + +4_{10} = 01011_2.$$

The resultant sum contains 5 bits. So, there is no carry out from sign bit. The sign bit '0' indicates that the resultant sum is **positive**. So, the magnitude of sum is 11 in decimal number system. Therefore, addition of two positive numbers will give another positive number.

Example 2

Let us perform the **addition** of two decimal numbers **-7** and **-4** using 2's complement method.

The **2's complement** representation of -7 and -4 with 5 bits each are shown below

$$-7_{10} = 11001_2$$

$$-4_{10} = 11100_2$$

The addition of these two numbers is

$$-7_{10} + -4_{10} = 11001_2 + 11100_2$$

$$\Rightarrow -7_{10} + -4_{10} = 110101_2$$

The resultant sum contains 6 bits. In this case, carry is obtained from sign bit. So, we can remove it

Resultant sum after removing carry is $-7_{10} + -4_{10} = 10101_2$

The sign bit '1' indicates that the resultant sum is **negative**. So, by taking 2's complement of it we will get the magnitude of resultant sum as 11 in decimal number system. Therefore, addition of two negative numbers will give another negative number.

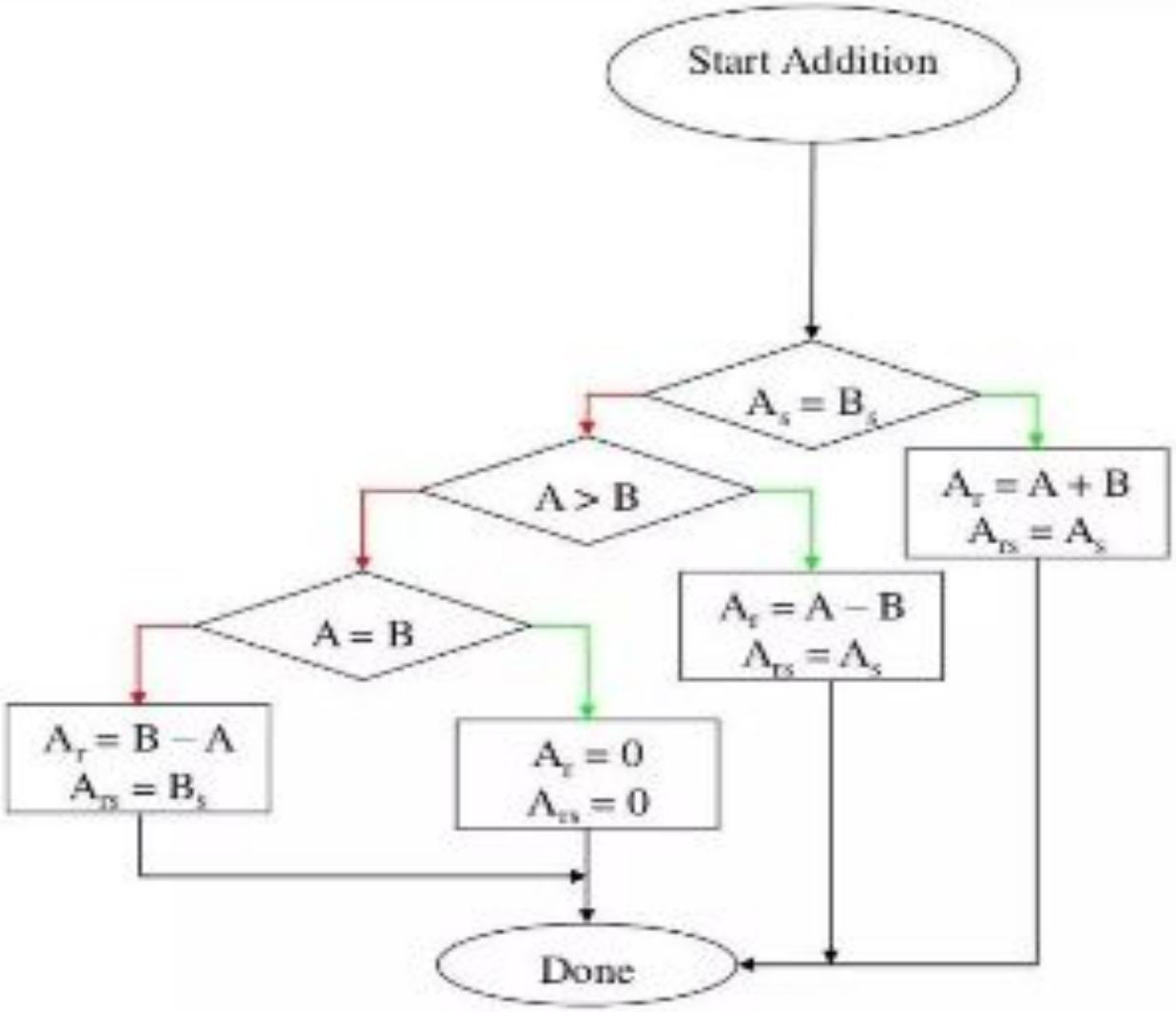
- Example of adding two magnitudes when the result is the sign of both operands:

	+3	0 011
+	+2	0 010
	+5	0 101

- Example of adding two magnitudes when the result is the sign of larger magnitude

	-3	1 011
+	+2	0 010
-(	+3	0 011
	- 2)	1 010
	-(1)	1 001

Flow Chart for Addition Operation



SUBTRACTION ALGORITHM

- When the sign of A and B are Different , add the magnitudes and attach the sign of A to the result.
- Otherwise compare the magnitudes and subtract the smaller number from the larger.
  - Choose the sign of result to be same as A if A>B
  - or the complement of sign of A if A<B
  - if A=B subtract B from A and make the sign of result positive

SIGNED BIT SUBTRACTION

Operation	Add Magnitudes	Subtract Magnitudes		
		A>B	A<B	A=B
(+A)-(-B)	+(A+B)			
(-A)-(+B)	-(A+B)			
(+A)-(+B)		+(A-B)	-(B-A)	+(A-B)
(-A)-(-B)		-(A-B)	+(B-A)	+(A-B)

Subtraction of two Signed Binary Numbers

Consider the two signed binary numbers A & B, which are represented in 2's complement form. We know that 2's complement of positive number gives a negative number. So, whenever we have to subtract a number B from number A, then take 2's complement of B and add it to A. So, **mathematically** we can write it as

$$A - B = A + 2's complement of B$$

Similarly, if we have to subtract the number A from number B, then take 2's complement of A and add it to B. So, **mathematically** we can write it as

$$B - A = B + 2's complement of A$$

So, the subtraction of two signed binary numbers is similar to the addition of two signed binary numbers. But, we have to take 2's complement of the number, which is supposed to be subtracted. This is the **advantage** of 2's complement technique. Follow, the same rules of addition of two signed binary numbers.

Example

Let us perform the **subtraction** of two decimal numbers **+7 and +4** using 2's complement method.

The subtraction of these two numbers is

$$+7_{10} - +4_{10} = +7_{10} + -4_{10}$$

The **2's complement** representation of +7 and -4 with 5 bits each are shown below.

$$+7_{10} - 00111_2$$

$$+4_{10} = 11100_2$$

$$\Rightarrow +7_{10} + +4_{10} = 00111_2 + 11100_2 = 00011_2$$

Here, the carry obtained from sign bit. So, we can remove it. The resultant sum after removing carry is

$$+7_{10} + +4_{10} = 00011_2$$

The sign bit '0' indicates that the resultant sum is **positive**. So, the magnitude of it is 3 in decimal number system. Therefore, subtraction of two decimal numbers +7 and +4 is +3.

Example

Let us perform the **subtraction of** two decimal numbers **+4** and **+7** using 2's complement method.

The subtraction of these two numbers is

$$+4_{10} - +7_{10} = +4_{10} + -7_{10}$$

The **2's complement** representation of +4 and -7 with 5 bits each are shown below.

$$+4_{10} = 00100_2$$

$$-7_{10} = 11001_2$$

$$\Rightarrow +4_{10} + -7_{10} = 00100_2 + 11001_2 = 11101_2$$

Here, carry is not obtained from sign bit. The sign bit '1' indicates that the resultant sum is **negative**. So, by taking 2's complement of it we will get the magnitude of resultant sum as 3 in decimal number system. Therefore, subtraction of two decimal numbers +4 and +7 is -3.

EXAMPLE

- Example of Subtracting two numbers with same sign bits

	+3	0 011
-	+2	0 010
	+1	0 001

- Example of Subtracting two numbers with Different sign bits

	-3	1 011
-	+2	0 010
-(	+3	0 011
	+2)	0 010
	-(5)	0 101

2's Complement

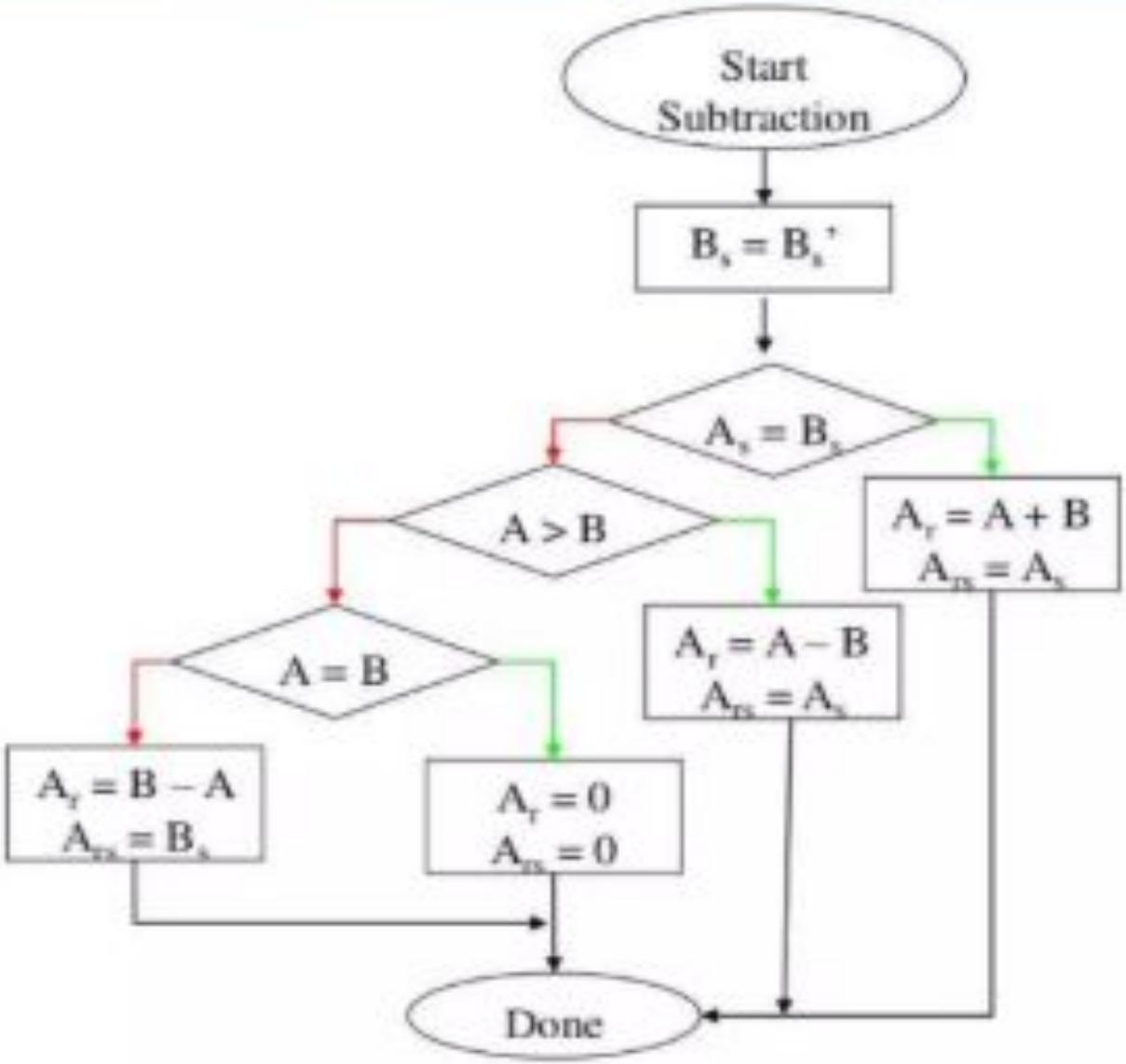
EXAMPLE

- Example of Subtracting two numbers with Different sign bits with 2's complement Arithmetic

+3 -> 0011
+2 -> 0010
-2 -> 2'scomp("+2")
-> 1110

	+3	0 011
-	+2	0 010
+(	+3	0 011
	-2)	1 110
	+(1)	0 001

Flow Chart for Subtract Operation



, DESIGN OF ADDER AND FAST ADDER

- A basic adder adds binary numbers using logic gates like half and full adders, while a fast adder, such as a carry-lookahead adder, uses more complex logic to predict and generate carries ahead of time, reducing propagation delay.
- Fast adders, like the carry-lookahead adder, achieve high speed by generating two signals, P (propagate) and G (generate), for each bit to pre-calculate carries for higher-order bits, significantly speeding up the operation compared to slower ripple-carry adders.

Adder design

- **Half Adder:** Adds two single bits, A and B, to produce a sum (S) and a carry-out (Cout).
  - **Logic:** $S = A \oplus B$ and $C_{out} = A \cdot B$.
  - **Implementation:** Typically uses one XOR gate for the sum and one AND gate for the carry.
- **Full Adder:** Adds three bits: two input bits (A, B) and a carry-in (Cin).
  - **Logic:**
    - Sum: $S = A \oplus B \oplus C_{in}$.
    - Carry-out: $C_{out} = (A \cdot B) + (B \cdot C_{in}) + (A \cdot C_{in})$.
  - **Implementation:** Requires two XOR gates, two AND gates, and one OR gate.
- **Ripple Carry Adder:** A multi-bit adder that links full adders in series, with the carry-out of one stage becoming the carry-in of the next. The carry signal "ripples" through the chain, making it the slowest type of adder because each stage must wait for the previous one to complete.

CARRY LOOK-AHEAD ADDER.

- The adder produce carry propagation delay while performing other arithmetic operations like multiplication and divisions as it uses several additions or subtraction steps.
- This is a major problem for the adder and hence improving the speed of addition will improve the speed of all other arithmetic operations.
- Hence reducing the carry propagation delay of adders is of great importance. There are different logic design approaches that have been employed to overcome the carry propagation problem.
- One widely used approach is to employ a carry look-ahead which solves this problem by calculating the carry signals in advance, based on the input signals. This type of adder circuit is called a carry look-ahead adder.

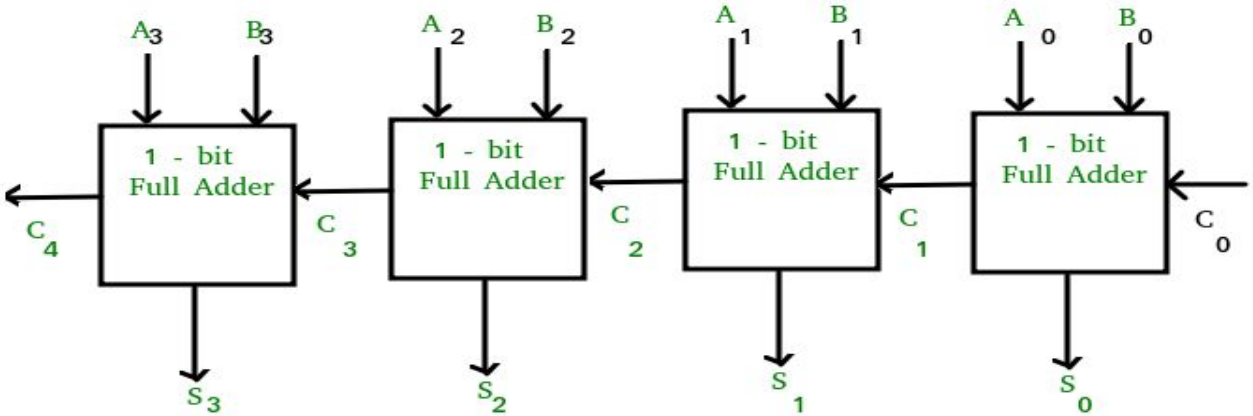
- **Here a carry signal will be generated in two cases:**
- Input bits A and B are 1
- When one of the two bits is 1 and the carry-in is 1.
- In ripple carry adders, for each adder block, the two bits that are to be added are available instantly.
- However, each adder block waits for the carry to arrive from its previous block.
- So, it is not possible to generate the sum and carry of any block until the input carry is known. The ith block waits for the $i-1th$ block to produce its carry. So there will be a considerable time delay which is carry propagation delay.

Fast adder design (Carry-Lookahead Adder)

- **Purpose:** To overcome the delay of the ripple carry adder by pre-calculating the carry for each stage.
- **Logic:**
  - **Generate (G) signal:** $G_i = A_i \cdot B_i$. A carry is generated at bit position i if both input bits (A_i and B_i) are 1.
  - **Propagate (P) signal:** $P_i = A_i \oplus B_i$. A carry is propagated through bit position i if at least one of the input bits is 1.
- **Carry calculation:** The carry-out (C_{i+1}) is calculated in a single step using the P and G signals and the carry-in (C_i).
  - **Equation:** $C_{i+1} = G_i + (P_i \cdot C_i)$.
- **Advantage:** By computing carries in parallel, it drastically reduces the delay compared to a ripple carry adder.
- **Disadvantage:** Requires more complex hardware (more logic gates) than a ripple carry adder, which can increase power consumption and area.

Other fast adder designs

- **Carry-Skip Adder:** Adds blocks of full adders and uses a carry-skip logic to jump over groups of bits if the carry-out is not needed in the next block.
- **Carry-Select Adder:** Generates two sums and two carries for each block (one assuming a carry-in of 0, and one assuming a carry-in of 1) and uses a multiplexer to select the correct one once the carry-in is known.
- **Parallel Prefix Adders:** These adders, such as the Kogge-Stone adder, use parallel carry trees to achieve a delay that grows logarithmically with the number of bits, making them very fast for large word sizes [2,



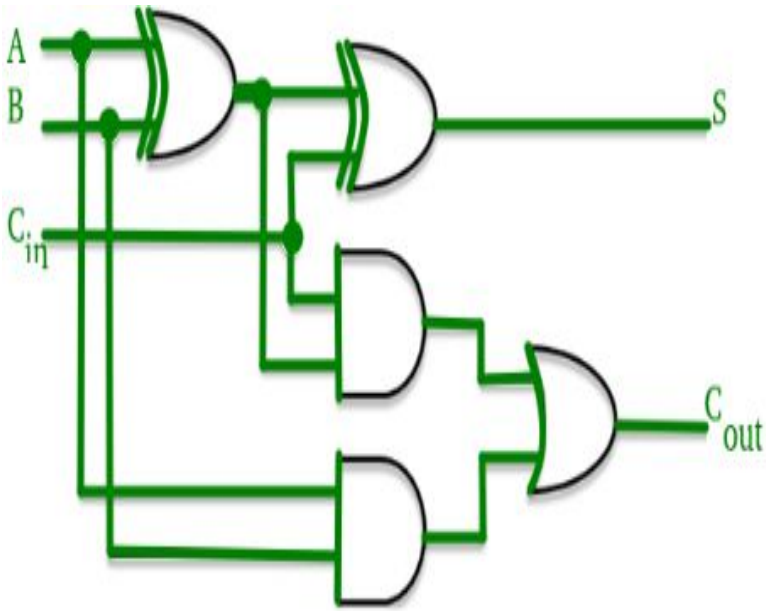
Consider the above 4-bit ripple carry adder. The sum *S*3 is produced by the corresponding full adder as soon as the input signals are applied to it. But the carry input *C*4 is not available on its final steady-state value until carry *C*3 is available at its steady-state value. Similarly *C*3 depends on *C*2 and *C*2 on *C*1 . Therefore, though the carry must propagate to all the stages in order that output S3 and carry C4 settle their final steady-state value.

The propagation time is equal to the propagation delay of each adder block, multiplied by the number of adder blocks in the circuit.

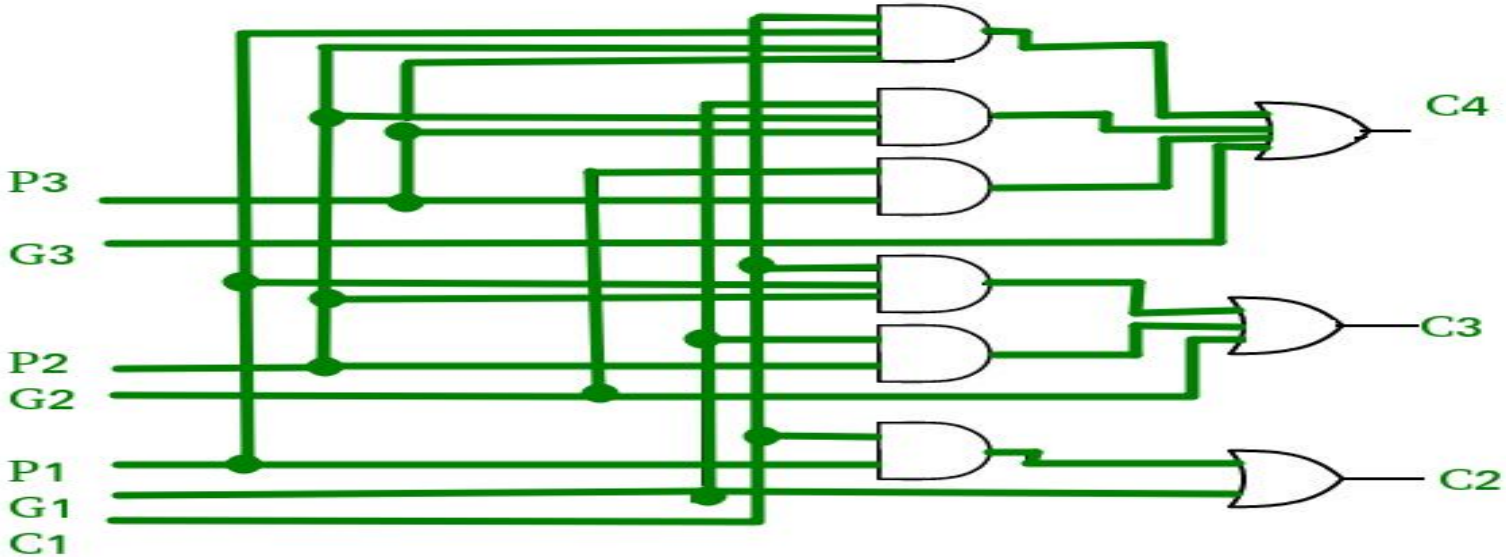
For example, if each full adder stage has a propagation delay of 20 nanoseconds, then S3 will reach its final correct value after 60 (20 × 3) nanoseconds. The situation gets worse, if we extend the number of stages for adding more number of bits.

A CARRY LOOK-AHEAD ADDER

- A carry look-ahead adder reduces the propagation delay by introducing more complex hardware. In this design, the ripple carry design is suitably transformed such that the carry logic over fixed groups of bits of the adder is reduced to two-level logic. Let us discuss the design in detail.



A	B	C	C +1	Condition
0	0	0	0	No Carry Generate
0	0	1	0	
0	1	0	0	
0	1	1	1	No Carry Propogate
1	0	0	0	
1	0	1	1	
1	1	0	1	Carry Generate
1	1	1	1	



The carry output Boolean function of each stage in a 4 stage carry look-ahead adder can be expressed as

$$C_1 = G_0 + P_0 C_{in}$$
$$C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_{in}$$
$$C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{in}$$
$$C_4 = G_3 + P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{in}$$

From the above Boolean equations we can observe that C_4 does not have to wait for C_3 and C_2 to propagate but actually C_4 is propagated at the same time as C_3 and C_2. Since the Boolean expression for each carry output is the sum of products so these can be implemented with one level of AND gates followed by an OR gate.

The implementation of three Boolean functions for each carry output (C_2, C_3 and C_4) for a carry look-ahead carry generator shown in below figure.

- Carry-lookahead addition is a fast way to add binary numbers by using logic to predict and generate carry bits simultaneously, instead of waiting for them to ripple through each position.
- For positive number multiplication, this is typically done by repeated addition, where a carry-lookahead adder can be used to speed up each step. Algorithms like the array multiplier and Booth multiplier can also be used, with carry-save addition used within the array multiplier to reduce carry-propagation delay.

CARRY LOOK AHEAD ADDITION, MULTIPLICATION OF POSITIVE NUMBERS

- Carry-lookahead addition is a fast way to add binary numbers by using logic to predict and generate carry bits simultaneously, instead of waiting for them to ripple through each position.
- For positive number multiplication, this is typically done by repeated addition, where a carry-lookahead adder can be used to speed up each step.
- Algorithms like the array multiplier and Booth multiplier can also be used, with carry-save addition used within the array multiplier to reduce carry-propagation delay.

Carry-lookahead addition

- **How it works:** A carry-lookahead adder (CLA) calculates carry bits in advance based on two signals for each digit position: a "generate" signal (G) and a "propagate" signal (P).
- **Generate (G):** Indicates that a carry will be generated by this position, regardless of the carry-in.
- **Propagate (P):** Indicates that a carry will be propagated from this position to the next, if a carry comes in.
- **Speed advantage:** By using logic to determine carries immediately, the CLA eliminates the ripple delay of simpler adders, making it significantly faster, especially for larger numbers.

- **Multiplication of positive numbers**
- **Repeated addition:** Multiplication can be viewed as repeated addition, where you add the multiplicand to itself the number of times indicated by the multiplier. A CLA can speed up each of these additions.
- **Array multiplier:** This method lays out the partial products in an array and adds them up.
  - **Carry-save addition:** To speed up the process, carry-save adders are used within the array multiplier. Instead of propagating the carry immediately, they save it to be added in a later stage, allowing partial sums to be calculated in parallel.
  - **Final addition:** A fast adder, such as a CLA, is used at the end of the array to perform the final addition of the partial products.

SIGNED OPERAND MULTIPLICATION

We now discuss multiplication of 2's-complement operands, generating a **double-length** product. The general strategy is still to accumulate partial products by adding versions of the multiplicand as selected by the multiplier bits.

First, consider the case of a **positive multiplier and a negative multiplicand**. When we add a negative multiplicand to a partial product, we must **extend the sign-bit value** of the multiplicand to the **left** as far as the product will extend. Figure shows an example in which a 5-bit signed operand, −13, is the multiplicand. It is multiplied by +11 to get the 10-bit product, −143. The sign extension of the multiplicand is shown in blue. The hardware discussed earlier can be used for negative multiplicands if it is augmented to provide for sign extension of the partial products.

```
 13   -> 1101
+13  -> 01101        [for +ve number, add 0 to MSB]
-13  -> 10010 +      [for –ve number, find 2's complement]
            1
     --------------
      10011  -> -13
```

[**To extend the sign bit** - since its 5 bit signed operand, 10 bit product should be generated. So, if the partial product's MSB is 1, add 1 for sign extension (to left),
if the partial product's MSB is 0, add 0 for sign extension (to left)]

Example: Sign extension of negative multiplicand

Sign extension is shown in blue

					1	0	0	1	1	(−13)
				×	0	1	0	1	1	(+11)
1	1	1	1	1	1	0	0	1	1	
1	1	1	1	1	0	0	1	1		
0	0	0	0	0	0	0	0			
1	1	1	0	0	1	1				
0	0	0	0	0	0					
1	1	0	1	1	1	0	0	0	1	(−143)

[product is10 bits ->(2n)]

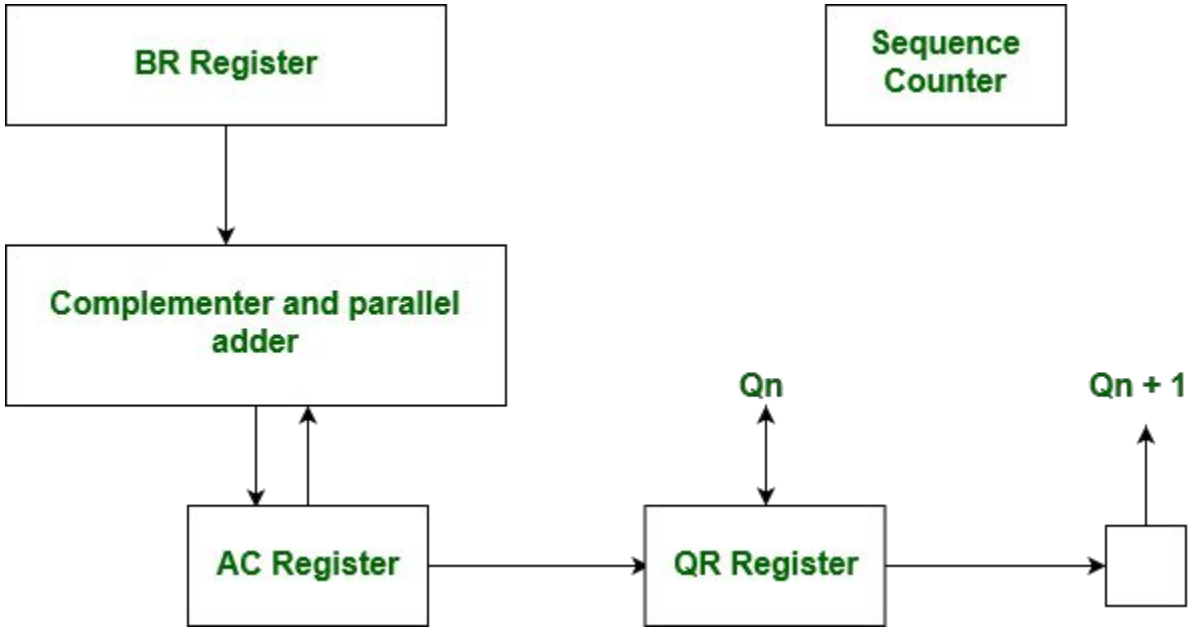
For a **negative multiplier**, a straightforward solution is to form the **2's-complement of both the multiplier and the multiplicand** and proceed as in the case of a positive multiplier. This is possible because complementation of both operands does not change the value or the sign of the product.

[If the sign bit is 0 then the number is positive, If the sign bit is 1, then the number is negative]

Booths algorithm for multiplying binary integers.

- Booth's algorithm is an efficient binary multiplication method for signed numbers in two's complement form that reduces the number of arithmetic operations.
- It works by examining pairs of bits in the multiplier to determine whether to add the multiplicand, subtract it, or do nothing, followed by a right shift of the accumulator and multiplier registers.
- This reduces operations by treating blocks of consecutive ones as a single operation.

- **Hardware Implementation of Booth's Algorithm**
- The hardware implementation of the booth algorithm requires the register configuration shown in the figure below:

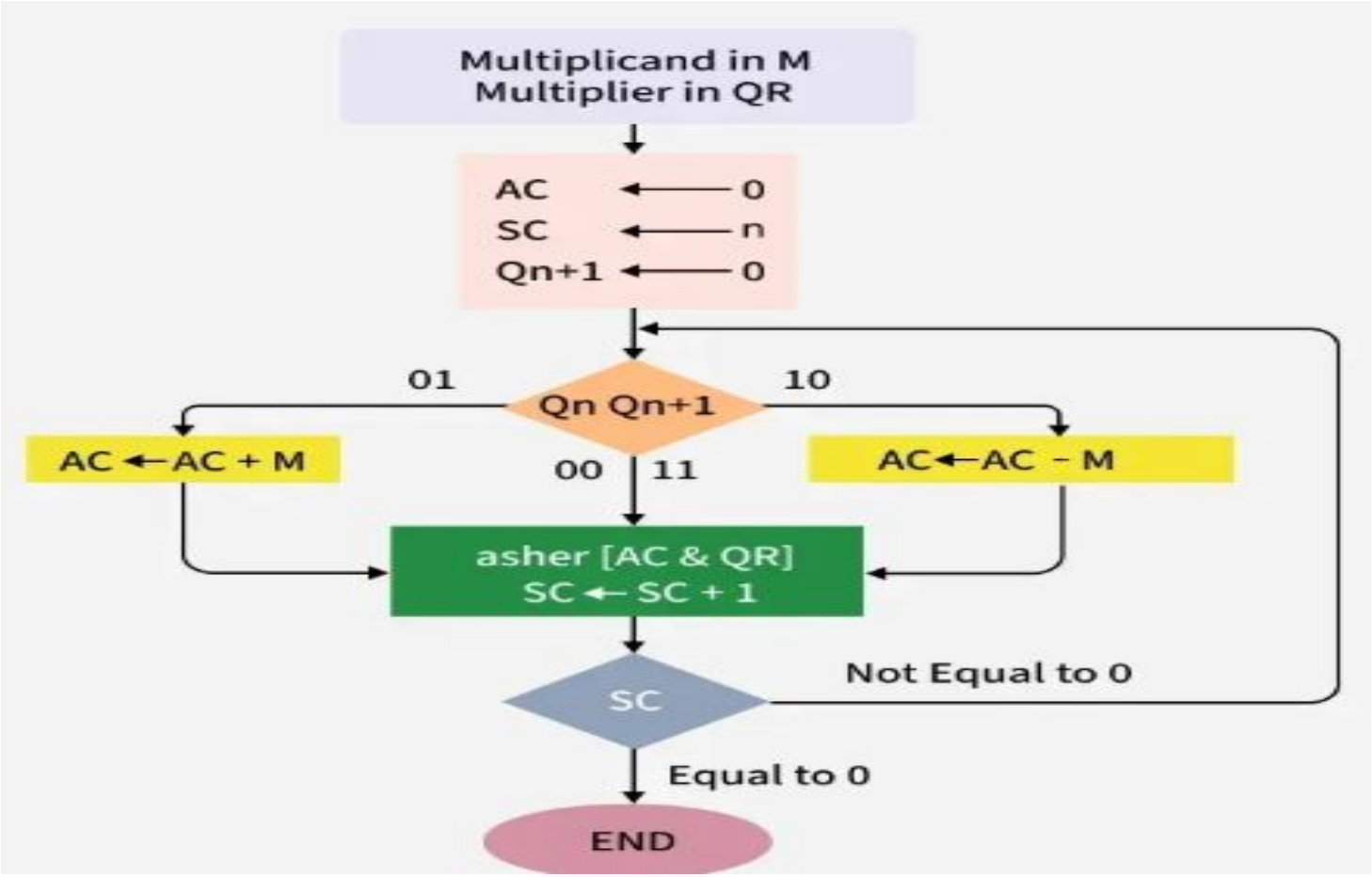


Booth's Algorithm Flowchart

We name the registers as A, B and Q, AC, BR and QR, respectively. Qn designates the least significant bit of the multiplier in the register QR.

An extra flip-flop Qn+1is appended to QR to facilitate a double inspection of the multiplier.

The flowchart for the booth algorithm is shown below:



- **Booth's Algorithm Steps**
- Initialize **AC = 0**, **Qn+1 = 0**, and set **SC = n** (number of multiplier bits).
- Check the two least significant bits (**Qn** and **Qn+1**).
- If bits = **10** → Subtract multiplicand (M) from AC (first 1 in a sequence).
- If bits = **01** → Add multiplicand (M) to AC (first 0 after ones).
- If bits = **00** or **11** → No change in AC (partial product unchanged).
- Perform an **Arithmetic Shift Right (ashr)** on [AC, QR, Qn+1] (sign bit in AC remains).
- Decrement sequence counter (SC).
- Repeat steps until SC = 0 (n iterations complete).
- Handle negative numbers: if multiplicand/multiplier is negative, take its **2's complement** to simplify operations.
- Final product obtained in [AC, QR] after all steps.

- **Example -** A numerical example of booth's algorithm is shown below for n = 4. It shows the step by step multiplication of -5 and -7.
- *BR = -5 = 1011,*
  BR' = 0100, <-- 1's Complement (change the values 0 to 1 and 1 to 0)
  BR'+1 = 0101 <-- 2's Complement (add 1 to the Binary value obtained after 1's complement)
  QR = -7 = 1001 <-- 2's Complement of 0111 (7 = 0111 in Binary)
  The explanation of first step is as follows: Qn+1
  AC = 0000, QR = 1001, Qn+1 = 0, SC = 4
  Qn Qn+1 = 10
  So, we do AC + (BR)'+1, which gives AC = 0101
  On right shifting AC and QR, we get
  AC = 0010, QR = 1100 and Qn+1 = 1

- **Initialize Registers:**
  - AC (Accumulator Register) ← 0
  - QR (Multiplier Register) ← Multiplier
  - BR (Multiplicand Register) ← Multiplicand
  - Qn+1 ← 0
  - SC (Sequence Counter) ← Number of bits in multiplier
- **Check Bit Pair (Qn, Qn+1):**
  - If (Qn, Qn+1) = (0, 1) → AC = AC + BR
  - If (Qn, Qn+1) = (1, 0) → AC = AC – BR
  - Else → No arithmetic operation
- **Perform Arithmetic Right Shift:**
  - Combine AC, QR, and Qn+1, then shift right by 1 bit.
- **Decrement SC:**
  - Reduce sequence counter by 1.
- **Repeat Steps 2–4:**
  - Continue until SC = 0.
- **Combine Results:**
  - The final product is found in the concatenated AC and QR registers.

Application of Booth's Algorithm

- **Processors and ALUs:** Booth's Algorithm enables efficient signed multiplication inside microchips and processors by reducing the number of additions/subtractions, leading to faster arithmetic logic unit (ALU) operations essential for computing, graphics, and cryptography.
- **Digital Signal Processing (DSP):** It accelerates multiplication tasks in DSP applications such as filtering and convolution for real-time audio, video, and signal processing.
- **Hardware Accelerators:** Specialized hardware for image processing, neural networks, and AI use Booth's Algorithm to speed up multiplication operations.
- **Cryptography:** Cryptographic operations involving large-number exponentiation benefit from faster multiplication with Booth's Algorithm, improving encryption and signature processes.
- **High-Performance Computing (HPC):** Large-scale scientific and mathematical computations employ Booth's Algorithm for optimized multiplication, enhancing overall system performance.
- **Embedded Systems:** Resource-limited embedded devices improve multiplication efficiency and power consumption by using Booth's Algorithm.
- **Network Packet Processing:** Booth's Algorithm helps efficient multiplication in network devices for operations on packet headers and payloads.

ADVANTAGES

- **Efficiency**: It reduces the number of additions and subtractions compared to standard multiplication, especially when there are long strings of ones or zeros in the multiplier.
- **Signed numbers**: It correctly handles multiplication of both positive and negative numbers by using two's complement representation.
- **Hardware implementation**: It is well-suited for hardware implementation because shifts are efficient operations in digital systems.

- The benefits of Booth's multiplication algorithm are:
- The algorithm is utilized to multiply signed binary numbers and it is effective for negative and positive numbers both without the requirement of individual logic for negative numbers.
- The algorithm reduces the number of additions and subtractions to be performed in multiplication and therefore the algorithm is efficient compared to other algorithms.
- Booth's algorithm is also highly hardware-friendly as it can reduce the arithmetic complexity of the operations significantly, which is invaluable when dealing with low-resource systems like embedded systems.
- The algorithm is efficient in dealing with strings of 0's or 1's in the multiplier by reducing shifting and operation to a minimum.
- It is mostly used in processors, digital signal processors (DSPs), and other hardware accelerators in an effort to maximize the acceleration of multiplication operations.

Thank You….